LAB 12

SSH Keys: Secure Authentication with GitHub

Windows Edition · Git Bash

■ Objective

By the end of this lab you will be able to generate an SSH key pair on Windows, add the public key to your GitHub account, and authenticate to GitHub without typing a username or password — ever again.

■ What is an SSH Key?

SSH (Secure Shell) uses a pair of cryptographic keys to prove your identity:

- **Private key** — stays on your computer, never shared.
- **Public key** — uploaded to GitHub. It is safe to share.

When you connect to GitHub, your computer signs a challenge with the private key. GitHub verifies it with the public key. No password is sent over the network.

■ Prerequisites

- Git for Windows installed (includes Git Bash)
- A GitHub account (github.com)
- Internet connection

■ Step-by-Step Guide

Step 1 Open Git Bash

Click the **Start menu**, search for **Git Bash**, and open it. You will see a terminal window with a $ prompt.

> ■ **NOTE:** Git Bash gives you a Linux-style terminal on Windows, which is needed to run SSH commands.

Step 2 Check for Existing SSH Keys

First, check whether you already have keys:

```
$ ls -al ~/.ssh
```

If you see files like **id_ed25519** or **id_rsa** — you already have keys. You can skip to Step 4. If you only see **known_hosts** or nothing at all, continue to Step 3.

Step 3 Generate a New SSH Key Pair

Run the command below, replacing the email with your GitHub email address:

```
$ ssh-keygen -t ed25519 -C "you@example.com"
```

You will be asked three questions. Here is what to do:

Prompt	What to do
Enter file in which to save the key	Press **Enter** to accept the default location
Enter passphrase (empty for no passphrase)	Press **Enter** to skip, or type a passphrase for extra security
Enter same passphrase again	Press **Enter** again (or re-type your passphrase)

After completion, two files are created in **~/.ssh/**:

- **id_ed25519** — your private key (keep this safe!)
- **id_ed25519.pub** — your public key (this goes to GitHub)

■ **TIP:** ed25519 is the modern, recommended key type. It is faster and more secure than the older RSA type.

Step 4 Start the SSH Agent

The SSH agent manages your keys in the background. Start it with:

```
$ eval "$(ssh-agent -s)"
```

You should see output like: **Agent pid 1234**

Now add your private key to the agent:

```
$ ssh-add ~/.ssh/id_ed25519
```

You should see: **Identity added: /c/Users/YourName/.ssh/id_ed25519**

Step 5 Copy Your Public Key

Display your public key so you can copy it:

```
$ cat ~/.ssh/id_ed25519.pub
```

You will see a long line starting with **ssh-ed25519** and ending with your email. Select the entire line and copy it (Ctrl+C or right-click → Copy).

■ **WARNING:** Copy the entire output — from ssh-ed25519 all the way to your email address. Do not add any spaces or line breaks.

Step 6 Add the Public Key to GitHub

Follow these steps on the GitHub website:

#	Action
1	Go to **github.com** and sign in
2	Click your profile photo (top-right) → **Settings**
3	In the left sidebar click **SSH and GPG keys**
4	Click the green button **New SSH key**
5	In the **Title** field, enter a name e.g. "My Windows Laptop"
6	In the **Key** field, paste your public key (Ctrl+V)
7	Click **Add SSH key** and confirm your password if asked

Step 7 Test the Connection

Back in Git Bash, test that everything is working:

```
$ ssh -T git@github.com
```

The first time you connect, you may see:

```
The authenticity of host 'github.com' can't be established.

Are you sure you want to continue connecting (yes/no)? yes
```

Type **yes** and press Enter. If successful you will see:

```
Hi YourUsername! You've successfully authenticated, but GitHub
```

■ **TIP:** The message says 'does not provide shell access' — that is normal and expected!

Step 8 Clone a Repository Using SSH

Now you can use SSH instead of HTTPS for all GitHub operations. On any GitHub repository page:

1. Click the green **Code** button
2. Select the **SSH** tab
3. Copy the URL (it starts with **git@github.com:**)

Then clone it in Git Bash:

```
$ git clone git@github.com:YourUsername/your-repo.git
```

■ **NOTE:** Always use the SSH URL (git@github.com:...) not the HTTPS URL (https://github.com/...) to benefit from key-based authentication.

Step 9 Change an Existing Repo from HTTPS to SSH

If you already cloned a repo with HTTPS and want to switch it to SSH:

```
# Check your current remote URL

$ git remote -v


# Change it to SSH

$ git remote set-url origin git@github.com:YourUsername/your-repo.git


# Verify the change

$ git remote -v
```

■ Quick Reference Cheat Sheet

Task	Command
Generate a new SSH key	`ssh-keygen -t ed25519 -C "you@example.com"`
Start SSH agent	`eval "$(ssh-agent -s)"`
Add key to agent	`ssh-add ~/.ssh/id_ed25519`
View public key	`cat ~/.ssh/id_ed25519.pub`
Test GitHub connection	`ssh -T git@github.com`
List SSH keys	`ls -al ~/.ssh`
Clone with SSH	`git clone git@github.com:User/repo.git`
Change remote to SSH	`git remote set-url origin git@github.com:User/repo.git`

■ Troubleshooting

Problem: Permission denied (publickey)

Solution: Your key is not added to GitHub or the agent. Re-run ssh-add and verify the key is on GitHub Settings → SSH keys.

Problem: ssh: connect to host github.com port 22: Connection timed out

Solution: Your network may block port 22. Try SSH over HTTPS port: edit ~/.ssh/config and add: Host github.com / Hostname ssh.github.com / Port 443

Problem: WARNING: UNPROTECTED PRIVATE KEY FILE

Solution: Your private key permissions are too open. Run: chmod 600 ~/.ssh/id_ed25519

Problem: Could not open a connection to your authentication agent

Solution: The SSH agent is not running. Run eval "$(ssh-agent -s)" first, then ssh-add.

■ Summary

1	Generated an Ed25519 SSH key pair on Windows using Git Bash

2	Started the SSH agent and added the private key
3	Uploaded the public key to GitHub
4	Tested the connection with ssh -T git@github.com
5	Cloned / switched repositories to use SSH authentication

■ **TIP:** You only need to do this setup once per computer. After that, all git push and git pull commands will authenticate automatically via SSH.